Lecture 1: Code, trust and responsability

Software Engineering under Generative AI

Madalina CROITORU

2026

Code with LLMs?

Noname manuscript No.
(will be inserted by the editor)

Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT

Burak Yetiştiren · Işık Özsoy · Miray Ayerdem · Eray Tüzün

the date of receipt and acceptance should be inserted later

Abstract

Context AI-assisted code generation tools have become increasingly prevalent in software engineering, offering the ability to generate code from natural language prompts or partial code inputs. Notable examples of these tools include GitHub Copilot, Amazon CodeWhisperer, and OpenAI's ChatGPT.

Objective This study aims to compare the performance of these prominent code generation tools in terms of code quality metrics, such as Code Validity, Code Correctness, Code Security, Code Reliability, and Code Maintainability, to identify their strengths and shortcomings.

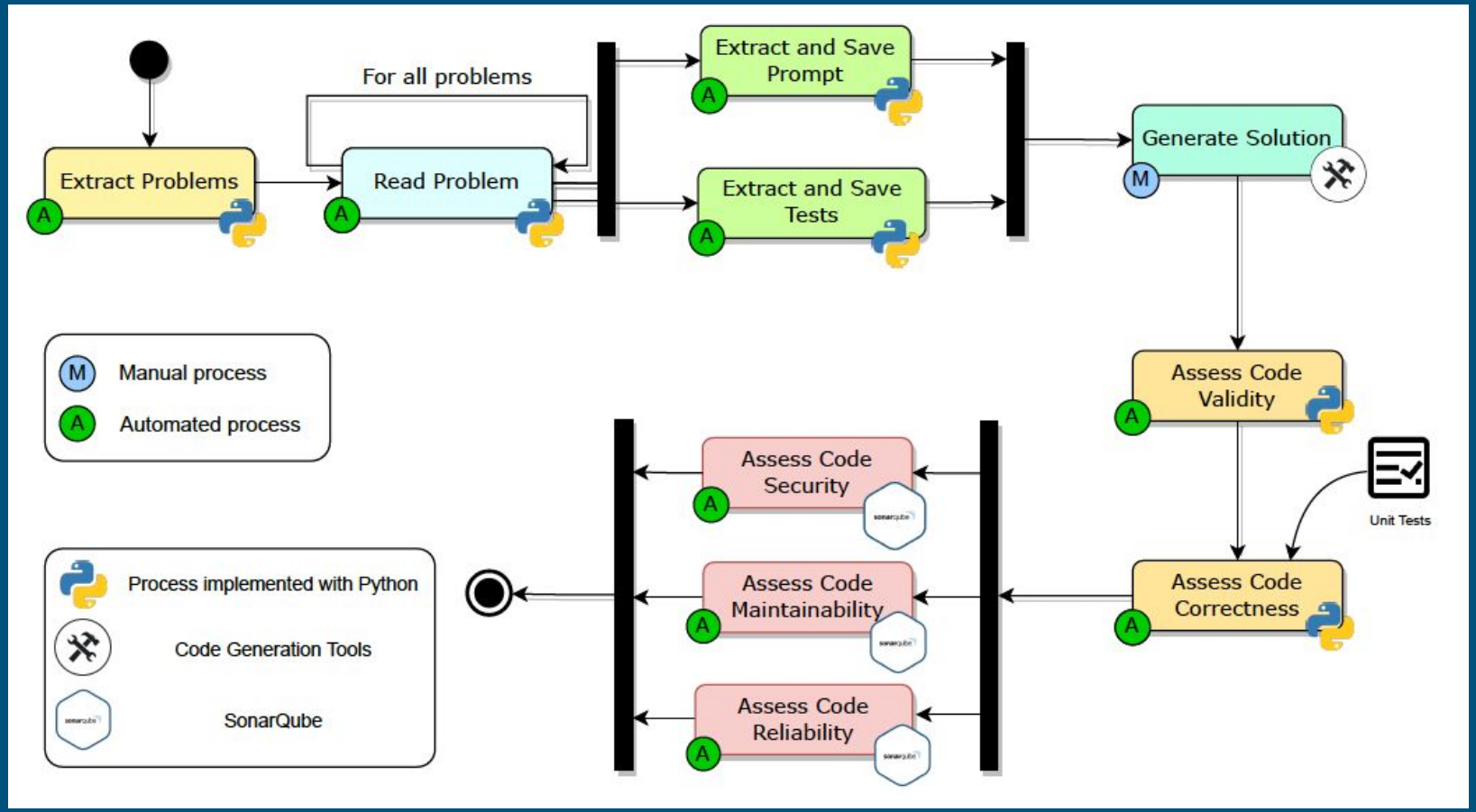
Method We assess the code generation capabilities of GitHub Copilot, Amazon CodeWhisperer, and ChatGPT using the benchmark HumanEval Dataset. The generated code is then evaluated based on the proposed code quality metrics.

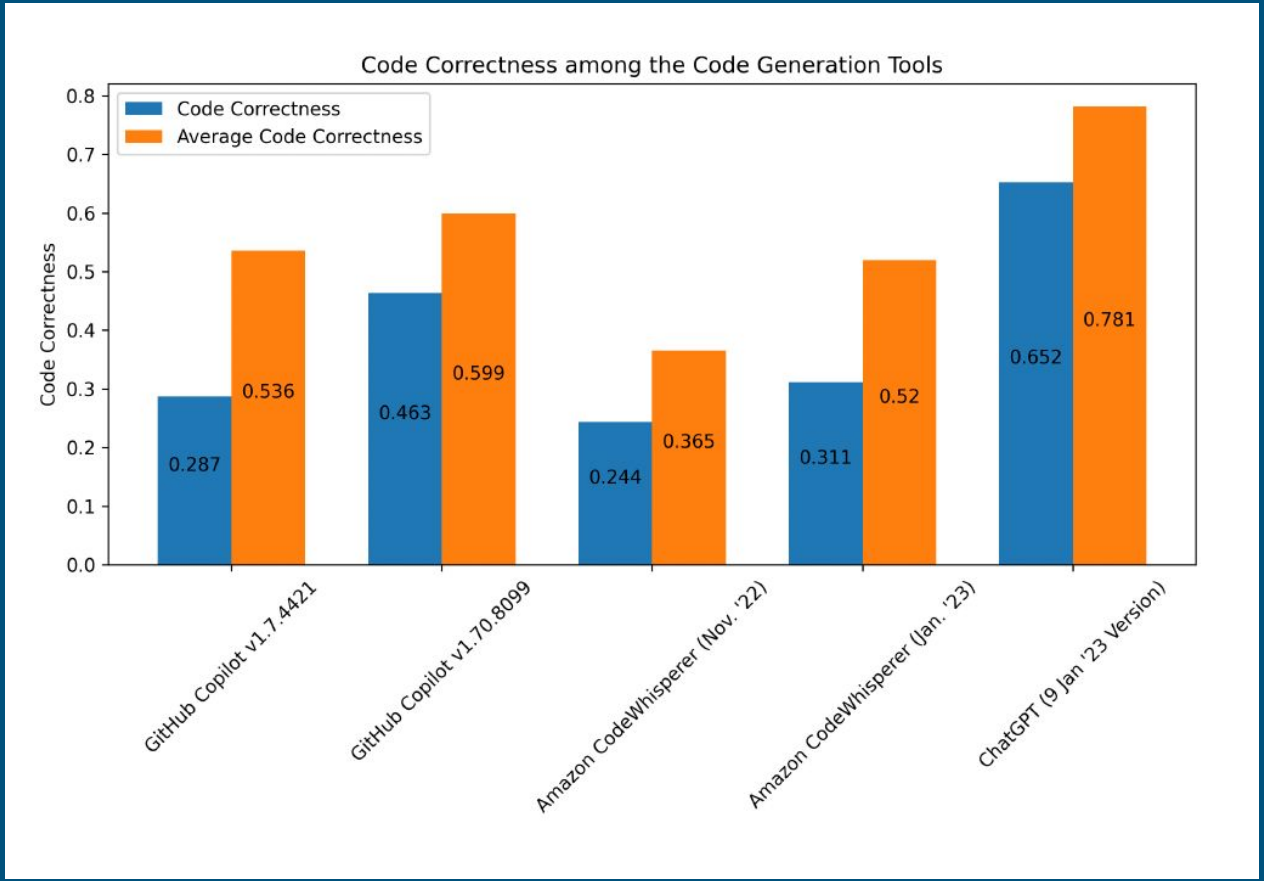
RQ1.1 How valid are the code generation tools' code suggestions?
RQ1.2 How correct are code generation tools' code suggestions?
RQ1.3 How secure are code generation tools' code suggestions?
RQ1.4 How reliable are code generation tools' code suggestions?
RQ1.5 How maintainable are code generation tools' code suggestions?

RQ2 What is the impact of using the docstrings on the generated code quality?

RQ3 What is the impact of using meaningful function names on the generated code quality?

RQ4 How did the code generation tools evolve over time?


For all problems
Extract Problems
Read Problem
Extract and Save Prompt
Extract and Save Tests
Generate Solution
Assess Code Validity
Unit Tests
Assess Code Correctness
Assess Code Security
Assess Code Maintainability
Assess Code Reliability
Manual process
Automated process
Process implemented with Python
Code Generation Tools
SonarQube


Code Correctness among the Code Generation Tools
Code Correctness
Average Code Correctness
Code Correctness
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.287
0.536
0.463
0.599
0.244
0.365
0.311
0.52
0.652
0.781
GitHub Copilot v1.7.4421
GitHub Copilot v1.70.8099
Amazon CodeWhisperer (Nov. '22)
Amazon CodeWhisperer (Jan. '23)
ChatGPT (9 Jan '23 Version)

LLM is Like a Box of Chocolates: the Non-determinism of ChatGPT in Code Generation

Shuyin Ouyang
University of Bristol
Bristol, United Kingdom

Jie M. Zhang
King's College London
London, United Kingdom

Mark Harman
University College London
London, United Kingdom

Meng Wang
University of Bristol
Bristol, United Kingdom

02828v1 [cs.SE] 5 Aug 2023

ABSTRACT

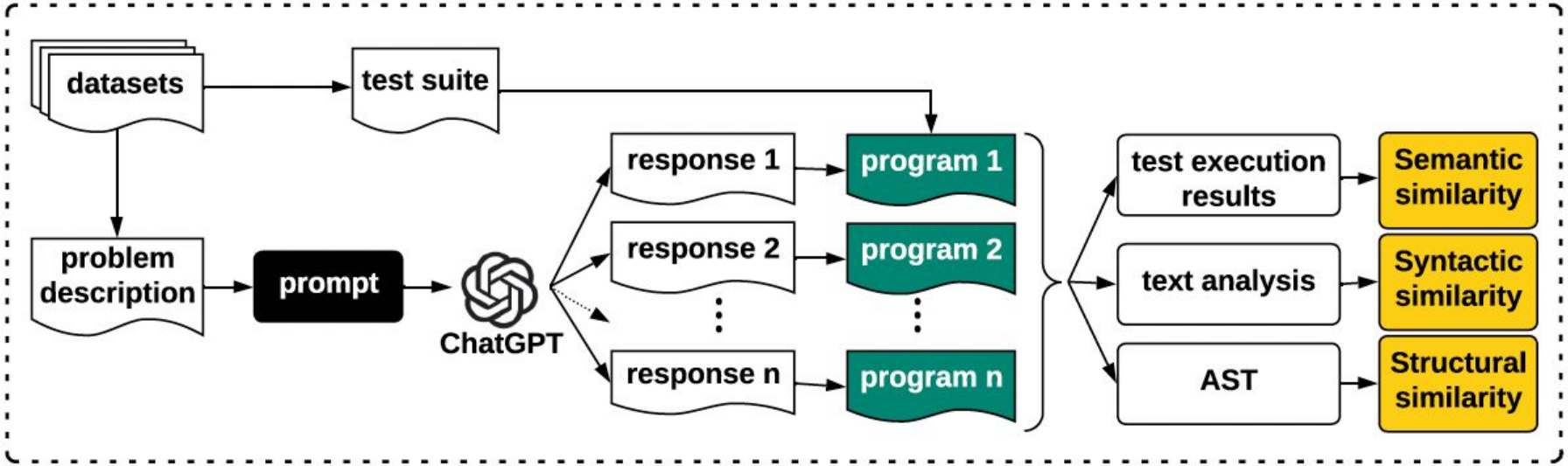
There has been a recent explosion of research on Large Language Models (LLMs) for software engineering tasks, in particular code generation. However, results from LLMs can be highly unstable; nondeterministically returning very different codes for the same prompt. Non-determinism is a potential menace to scientific conclusion validity. When non-determinism is high, scientific conclusions simply cannot be relied upon unless researchers change their behaviour to control for it in their empirical analyses. This paper conducts an empirical study to demonstrate that non-determinism is, indeed, high, thereby underlining the need for this behavioural change. We choose to study ChatGPT because it is already highly prevalent in the code generation research literature. We report results from a study of 829 code generation problems from three code generation benchmarks (i.e., CodeContests, APPS, and HumanEval). Our results reveal high degrees of non-determinism: the ratio of coding tasks with zero equal test output across different requests is 72.73%, 60.40%, and 65.85% for CodeContests, APPS, and HumanEval, respectively. In addition, we find that setting the temperature to 0 does not guarantee determinism in code generation, although it indeed brings less non-determinism than the default

LLMs when completely different suggestions are given at different times [53].

Moreover, non-determinism affects the reliability and reproducibility of empirical software engineering [43]. Indeed, compared to other tasks of ChatGPT, such as question answering and text summary, the non-determinism threat in code-related tasks is much more serious, because the inconsistency (especially semantic inconsistency) often suggests errors in the generated code [23]. It is therefore of vital importance to understand how serious the non-determinism is for LLM-based software engineering tasks.

This paper presents the first systematic empirical study on the threat of non-determinism of ChatGPT in code generation tasks. We choose the code generation task because code generation with Large Language Models (LLMs), such as ChatGPT, has recently attracted significant attention due to its impressive and cutting-edge performance [9, 29]. Many publications have emerged from both the software engineering community and the machine learning community on evaluating the capability of ChatGPT in code generation [5, 9, 14, 33, 57].

We conduct a series of experiments using the ChatGPT models on three widely-studied code generation benchmarks (i.e. CodeContests, APPS, and HumanEval) with 829 coding problems. For each


datasets
test suite
problem description
prompt
ChatGPT
response 1
response 2
response n
program 1
program 2
program n
test execution results
text analysis
AST
Semantic similarity
Syntactic similarity
Structural similarity

RQ1: To what extent is ChatGPT susceptible to non-determinism in code generation? This RQ investigates the non-determinism of ChatGPT in terms of the semantic, syntactic, and structural similarity among the code candidates generated with identical instructions. There are three sub-RQs:

- *Sub-RQ1.1: To what extent is ChatGPT susceptible to non-determinism in terms of semantic similarity?*
- *Sub-RQ1.2: To what extent is ChatGPT susceptible to non-determinism in terms of syntactic similarity?*
- *Sub-RQ1.3: To what extent is ChatGPT susceptible to non-determinism in terms of structural similarity?*

RQ2: How does temperature affect the degree of non-determinism Temperature is a hyperparameter of LLMs for controlling the randomness of the predictions. This RQ checks and compares the non-determinism of ChatGPT in code generation with different choices of temperature.

RQ3: How does the non-determinism compare to the similarity of the top code candidates generated within the same prediction? ChatGPT can be configured to generate multiple candidates for one prediction, which are ranked by their predictive probability. This RQ compares the similarity of the code candidates obtained in different predictions with those obtained within the same prediction.

RQ4: Is there any correlation between different code generation tasks and the non-determinism degree? To understand what affects non-determinism, this RQ studies the correlation between code generation features (e.g., the length of code generation instructions and the code generation difficulty) and the similarity metric uesed in our study.

Answer to RQ1.1: The semantic difference among the code generated by ChatGPT in different requests is significant. In particular, the ratio of coding tasks with not a single equal test output among the five different requests is 72.73%, 60.40%, and 65.85% for CodeContests, APPS, and HumanEval, respectively. In addition, the maximum difference of the test pass rate reaches 1.00 for all three datasets and accounts for 48.17% of the problems in HumanEval, the most widely used code generation benchmark.

Answer to RQ1.2: Code candidates generated by ChatGPT in different requests also differ significantly in syntax. The mean syntax similarity (LCS) is only 0.12, 0.14, and 0.45 for CodeContests, APPS, and HumanEval, respectively.

Answer to RQ2: Contrary to the widely held belief (and common practices), setting temperature to 0 does not guarantee determinism in code generation, although it indeed brings better determinism than the default configuration (*temperature*=1) for all three types of similarities. Setting the temperature to 2 tends to yield uniformly worse code candidates, which may perversely improve the similarity among code candidates.

What is the conclusion?

Trust I

- Software evolution: change is inevitable and that systems degrade unless actively controlled.
- From this perspective, the key question is not whether software works at a given moment, but whether its evolution remains governable over time.
- Architectural and modular structures determine where decisions are made and where constraints are enforced, which in turn determines where responsibility lies.
  - When decisions are scattered or implicit, responsibility becomes diffused, making it difficult to attribute faults or manage change in a disciplined way.

Trust II

- Classical software engineering does not define *trust* as a “psychological” state, but it does define the conditions under which a developer can reasonably accept responsibility for a piece of code.
- Good modular design allows a programmer to change part of a system without understanding the entire system, thereby limiting the consequences of change:
- Responsibility is inseparable from predictability: a developer can accept ownership of code only if they can anticipate how changes propagate through it.

Trust and GenAI

- Generative tools can produce functionally correct code at scale, but they do not guarantee decision locality, explicit assumptions, or controlled change impact.
- As a result, developers are increasingly confronted with code that works but for which responsibility is difficult to justify.

Agile Game 1: responsibility in classic software engineering

Game I - general rules

- Each row: a team
- Each team must produce EXACTLY 10 CORRECT sentences
- Sentence rules:
  - EXACTLY 6 words
  - MUST include a noun, a verb and an adjective
  - MUST be grammatically correct
- Example: “Small robots learn difficult social rules.”

Game I - specific roles

A team:

2-3 sentence creators

2-3 checkers (verify the rules and check with creators)

2-3 approvers (make the final decisions)

1 recorder (write down the sentences)

Game 1 - round 1: 10 minutes

Creators propose sentences verbally. Checkers say “OK” or “NOT OK”. Approvers accept or reject.

Recorder writes accepted sentences. Rejected sentences are discarded and must be recreated from scratch.

Game 1 - mid-Game Decision Change

Checkers are removed. Approvers must check rules themselves. Creators may not explain their sentences.

A new round of 10 minutes to produce 10 new phrases

Game I: conclusion

Who felt responsible for what in Round 2?

What that responsibility implicit or assumed?

Did your behaviour change (i.e. were you more careful after Round 2?)?

Agile Game 2: enter GenAI

Game II - general rules

- Each row: a team
- Each team must produce EXACTLY 8 rules on **how to run a fair group project**
- The rules:
  - ONE sentence per rule
  - Enforcable
  - MUST not contradict another rule
- Example: “Conflicts should be resolved quickly to avoid delays.”

Game II - roles

- 3-4 Writers – propose human-written rules
- 2-3 Reviewers – check clarity and consistency
- 1-2 Integrators – decide inclusion, modification, or rejection
- 1 Recorder – writes final rules and marks origin (H / AI / AI+H)

Game II - round 1 - 10 minutes

Writers propose rules.

Reviewers check them.

Integrators decide.

Recorder writes accepted rules.

(No AI input)

AI Injection - every 90 seconds

For each AI rule, integrators must choose: accept as-is, modify, or reject. Recorder must mark the origin of each accepted rule.

Production never stops.

- All team members must contribute equally at all times.
- Deadlines may be adjusted if most members feel stressed.
- One person should have final authority to resolve conflicts.
- All decisions must be documented, unless time is limited.
- Peer evaluation should influence individual grades.
- Silence during meetings counts as agreement.
- Late work is acceptable if the reason is reasonable.
- Tasks should be reassigned when progress is slow.
- Major decisions should aim for consensus when possible.
- If a rule is unclear, the project lead decides its meaning.
- Feedback should be constructive but not mandatory.
- Members may work independently as long as results align.
- Conflicts should be resolved quickly to avoid delays.
- Responsibility lies with the group, not individuals.
- Rules may be revised if they no longer fit the situation.
- Efficiency is more important than equal workload distribution.
- Meetings should be optional unless critical decisions are needed.
- Quality matters more than speed, except near deadlines.
- Disagreements should be escalated only as a last resort.
- Every member is accountable for the final outcome.

Removing AI rules becomes costly

For each accepted AI rule to be removed two further human rules must be removed.

End state: exactly 8 rules with origins clearly marked

Discussion Game 2

- Which AI rules were accepted too quickly?
- Who felt **responsible** for AI-originated problems?
- Did AI change the pace or the quality of decisions?

Agile Game 3

Game III - roles

Each pair incrementally builds a set of rules describing how a robot should behave in a classroom.

- Rules are treated as executable constraints (similar to code).
- One sentence per rule
- Must be precise enough to enforce
- Must not contradict existing rules

Students form pairs. Each pair takes one sheet and writes the initial rules:

1. The robot must not harm humans.

2. The robot must follow teacher instructions.

Game III: a round

- A round lasts 5 minutes
- During a round each pair will:
  - Add one new rule, or
  - Modify one existing rule (with a one-line justification).
- At the end of each round, pairs pass their sheet to the next pair. No verbal or written explanations
- may be given during handover.
- 6 rounds will take place.

Game III: decision change

Deleting rules is now forbidden - pairs must adapt without deletion

5 more rounds

Some groups read outloud the rules set

Questions:

- Which rules were hardest to change later?
- Did you trust rules you did not write?
- Who felt responsible for contradictions or problems?

What does it mean for code?

Example code

- The code implements a simple decision procedure for processing applications.
- Each application contains basic attributes such as age, score, submitted documents, and a referral flag.
- The procedure evaluates a fixed set of rules to decide whether the application should be approved or rejected, and it returns both the decision and a reason code.
- The rules are applied in a specific order.
  - Applications are immediately rejected if the applicant is under 18 years old or if required documents are missing.
  - If those checks pass, the application is approved when the score meets a given threshold, or when a slightly lower score is compensated by a referral.
  - All other cases are rejected.
- After a decision is made, the system logs the final decision and its reason.
- Both implementations produce the same outputs for the same inputs and satisfy the same tests.

Change-impact justification

Q1. For *Version H*, list the exact functions or code locations that must be modified to implement the change correctly.

Q2. For *Version AI*, list the exact functions or code locations that must be modified to implement the same change.

Q3. For each version, explain whether you can justify that no other locations should be affected.

Q4. Based on your analysis, which version would you be willing to accept responsibility for modifying, and why?

Failure traceability

Q5. For *Version H*, identify the most plausible code location(s) where the failure could originate.

Q6. For *Version AI*, identify the most plausible code location(s) where the failure could originate.

Q7. For each version, explain whether the failure could be reasonably anticipated from the code structure alone.